

NodeScope

Interactive Virtual LAN Simulator and Network Troubleshooting Lab

Python 3.11+ / Flask | SQLite Relational Engine | Vanilla JS + SVG Topology | 18/18 Pytest Verified | 100% Simulation-Driven

Course:	**Computer Networks & Distributed Systems**
Project Type:	**Computer Engineering Mini-Project**
Technology Stack:	**Python, Flask, SQLite, Bootstrap 5, SVG**
Repository:	**github.com/Kavishkhan15/NodeScope**
Academic Year:	**2026 – Final Examination Defense**

Department of Computer Engineering & Information Technology

Laboratory Practical & Viva Voce Project Documentation

1. Executive Summary & Abstract

NodeScope is an interactive, simulation-driven educational virtual Local Area Network (LAN) platform engineered specifically for undergraduate Computer Engineering and Information Technology students. Traditional Computer Networks education suffers from a persistent disconnect between theoretical textbook concepts (such as CIDR subnetting, Address Resolution Protocol, the DHCP DORA sequence, and ICMP ping diagnostics) and hands-on laboratory experience. Physical networking hardware is costly and difficult to reconfigure safely, while commercial emulation suites such as Cisco Packet Tracer demand significant system resources and emphasize vendor-specific Command Line Interface (CLI) memorization over fundamental troubleshooting logic.

NodeScope eliminates these hurdles by delivering a **zero-configuration, web-based laboratory** executable on standard Windows workstations using Python 3 and modern web standards. Unlike physical network sniffers or Scapy-based utilities that require root/administrator privileges and physical packet injection, NodeScope models private IPv4 local area networks **100% deterministically in application software**. The platform features an SVG-driven drag-and-drop workspace, automatic IEEE-compliant MAC generation, dynamic DHCP lease pool management, Layer 2 to Layer 3 Virtual ARP Tables, Breadth-First Search (BFS) topological packet forwarding verification, and an intelligent **Guided IP Address Conflict Resolver**.

> **Key Innovation Highlight:** When an IP address collision occurs, NodeScope dynamically detects the conflict across active interfaces, highlights affected devices with real-time pulsing red collision halos, marks the corresponding ARP entry as `AMBIGUOUS (IP Conflict)`, and computes the lowest available unassigned candidate IP in the subnet to guide students through real-time resolution.

2. Problem Statement & Comparative Analysis

In academic networking laboratories, students must grasp how packets traverse switches and how common misconfigurations—such as duplicate static IPs or disconnected patch cables—impair LAN communication.

Feature / Dimension	Physical Hardware Lab	Cisco Packet Tracer	Wireshark / Scapy	NodeScope (Our Project)
Cost & Hardware Requirements	High (Routers, Switches, NICs, Patch Cords)	Free with NetAcad account (500MB installer)	Free, but requires physical LAN traffic	**Zero cost, 100% portable Python application**
Privilege Requirements	Physical console access	Standard user	Requires WinPcap / Npcap & Administrator privileges	**Zero root/admin privileges needed**
IP Conflict Visualization	Silent failure or OS popup	Generic packet drop	Requires packet inspection analysis	**Pulsing red visual halos & ambiguous ARP flag**
Guided Conflict Resolution	Manual trial & error	Manual CLI commands	None (analyzer only)	**Automated candidate IP scan & 1-click fix**

Feature / Dimension	Physical Hardware Lab	Cisco Packet Tracer	Wireshark / Scapy	NodeScope (Our Project)
Extensibility & Codebase	Hardware firmware (Closed)	Proprietary (Closed Source)	Open Source C/Python library	**Open, readable Python Flask + SQLite architecture**

3. System Architecture & Relational Data Model

NodeScope adopts a decoupled Model-View-Controller (MVC) architecture coupled with a pure Python simulation engine. All network states—including topologies, device coordinates, dynamic leases, and event histories—are persisted in an embedded SQLite database.



Relational Database Schema

The underlying SQLite schema maintains foreign-key integrity (`PRAGMA foreign_keys = ON;`) to guarantee that when a device or network is deleted, all dependent links, leases, and conflicts cascade cleanly:

- `networks`: Stores virtual LAN definition, subnet CIDR (e.g. `192.168.10.0/24`), derived netmask, gateway, and DHCP bounds.
- `devices`: Stores device type (router, switch, desktop, etc.), power state (`is_online`), approval state (`is_approved`), mode (`dhcp` vs `static`), assigned IPv4, MAC address, and canvas coordinates (`pos_x`, `pos_y`).
- `connections`: Stores bidirectional link edges (`device_a_id`, `device_b_id`) and physical link type (`wired` vs `wireless`).
- `dhcp_leases`: Tracks dynamic host configuration state, MAC bindings, lease granted timestamps, and release statuses.
- `ip_conflicts`: Real-time registry of detected address collisions and resolutions.
- `network_events`: Persistent timestamped audit stream for security alerts, lease updates, and diagnostic outcomes.

4. Core Simulation Algorithms

4.1 Subnetting & Addressing Engine (`simulator/addressing.py`)

All IPv4 subnet operations are governed strictly by Python's built-in `ipaddress` module:

- **Subnet Validation:** Verifies that user-provided subnets conform to RFC 1918 private space and valid prefix lengths (`/8` to `/30`).
- **Boundary Constraints:** Prevents assignment to the Network Address (host bits = 0) or Directed Broadcast Address (host bits = 1).
- **Candidate Discovery Algorithm:**

```
def find_available_ip(network_cidr, assigned_ips):
    net = ipaddress.ip_network(network_cidr)
    for host in net.hosts():
        ip_str = str(host)
        if ip_str not in assigned_ips and ip_str != net_gateway:
            return ip_str # Returns lowest unused candidate IPv4
    return None # Pool exhaustion
```

4.2 Deterministic ICMP Ping Pipeline (`simulator/ping.py`)

Rather than generating random packet loss, the ICMP Ping Simulator evaluates a deterministic 5-step diagnostic chain modeled after standard networking failure modes:

1. **Source State Verification:** Source device must be powered on (`is_online == 1`) and configured with an IP.
2. **Destination State Verification:** Destination device must be powered on and assigned an IP.
3. **Subnet Match:** Evaluates bitwise whether both hosts reside within the identical CIDR network prefix.
4. **Address Uniqueness Check:** Verifies that the destination IPv4 address is not involved in an active address collision (simulating the duplicate ARP reply dilemma).
5. **Topological BFS Graph Reachability:** Constructs an adjacency graph from the `connections` table and executes Breadth-First Search (BFS). Packet forwarding succeeds only if an unbroken sequence of powered-on switches or wireless access points connects the endpoints.

4.3 Intelligent IP Conflict Detection & Resolution (`simulator/conflicts.py`)

When an IP address is saved, the collision engine queries all active interfaces:

```
SELECT ip_address, COUNT(*) as count
FROM devices
WHERE network_id = ? AND is_online = 1 AND ip_address IS NOT NULL
GROUP BY ip_address HAVING count > 1;
```

If a duplicate is identified:

1. The conflict is recorded in `ip_conflicts` .
2. Colliding nodes are assigned a CSS `conflict-halo` (pulsing red animation).

3. The Virtual ARP Table sets status to `AMBIGUOUS (IP Conflict)`.
4. The administrator can launch the **Guided Conflict Resolver**, which suggests the lowest free candidate IP, updates the target device, and instantly clears the fault.

5. Visual Application Tour

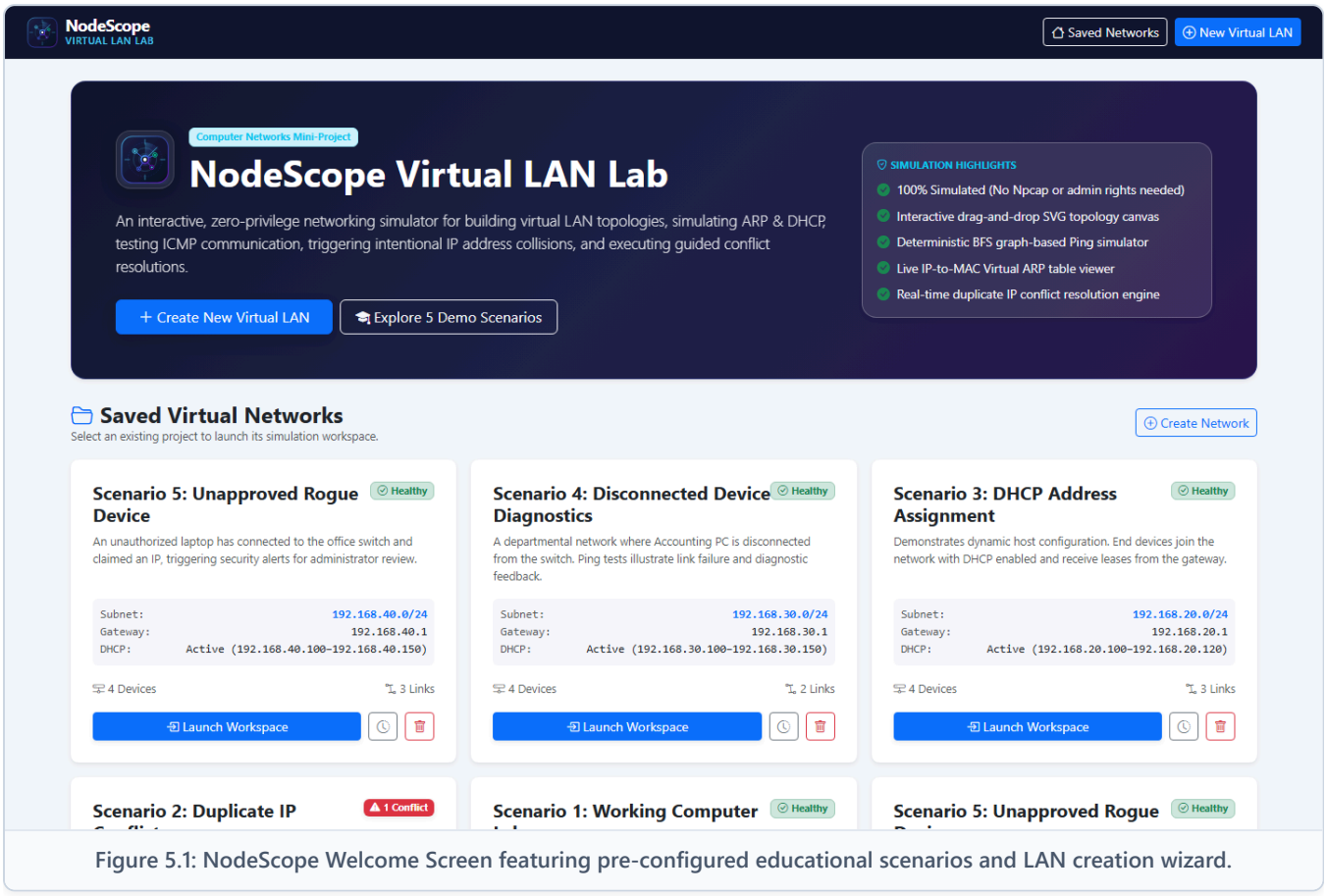


Figure 5.1: NodeScope Welcome Screen featuring pre-configured educational scenarios and LAN creation wizard.

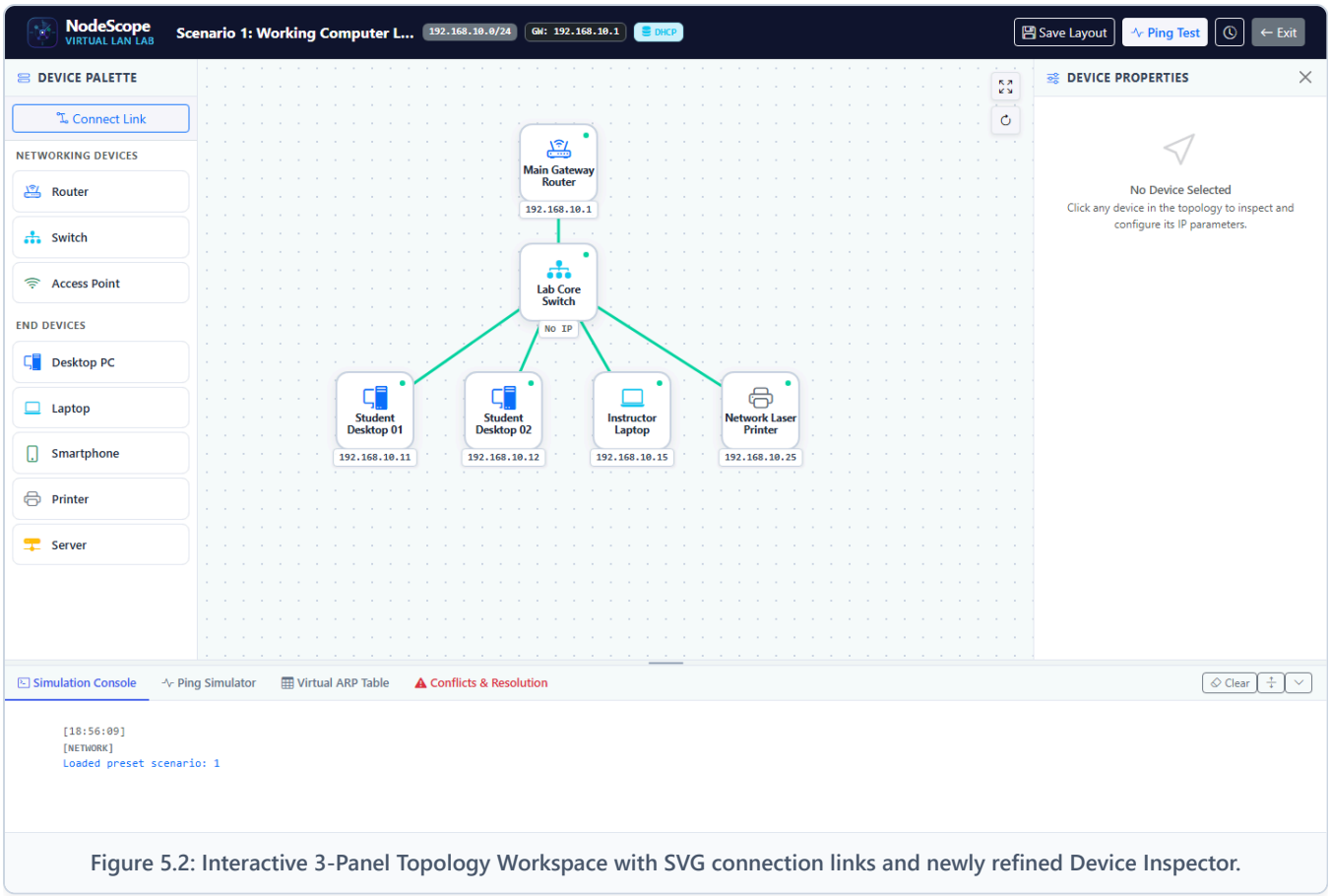


Figure 5.2: Interactive 3-Panel Topology Workspace with SVG connection links and newly refined Device Inspector.

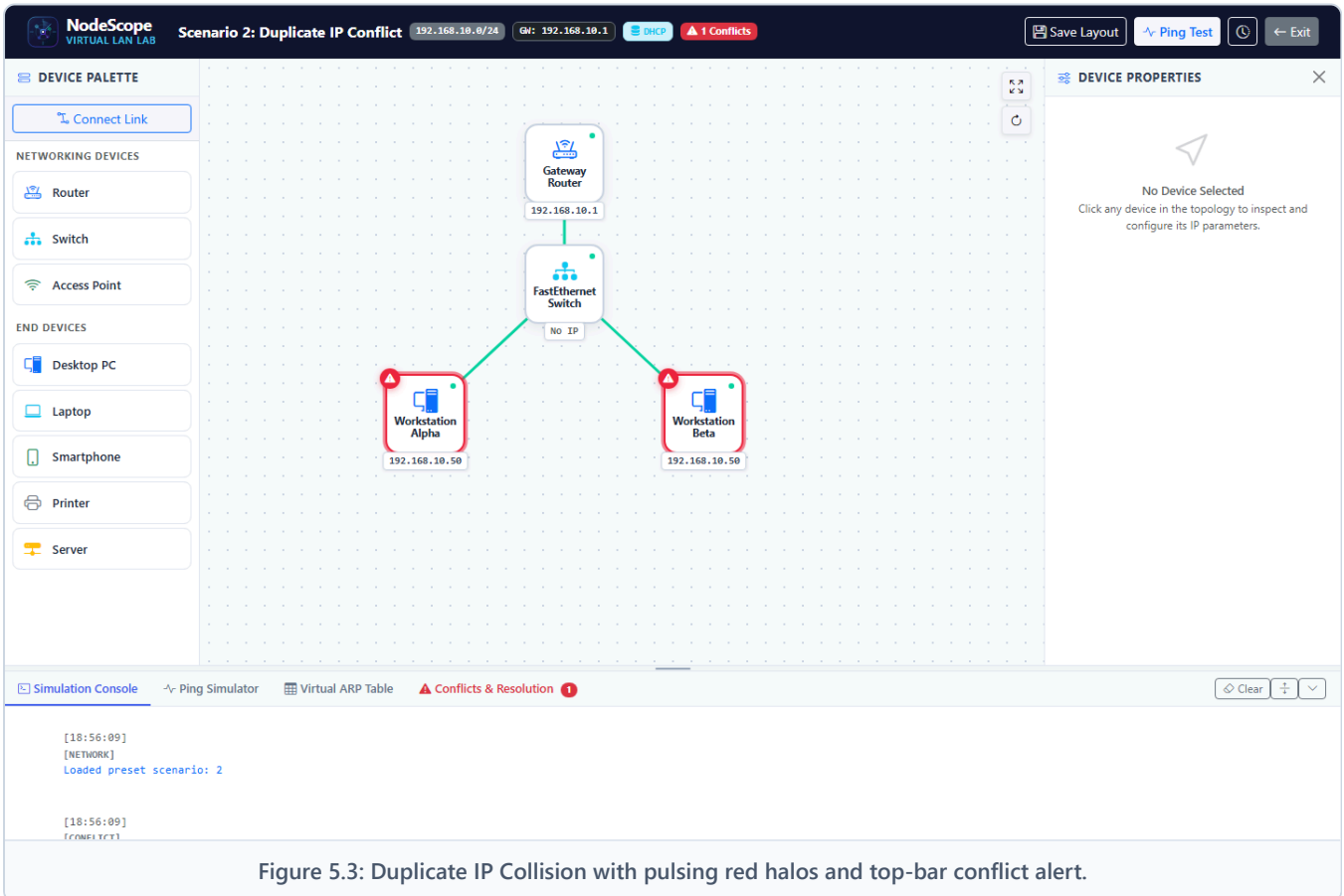


Figure 5.3: Duplicate IP Collision with pulsing red halos and top-bar conflict alert.

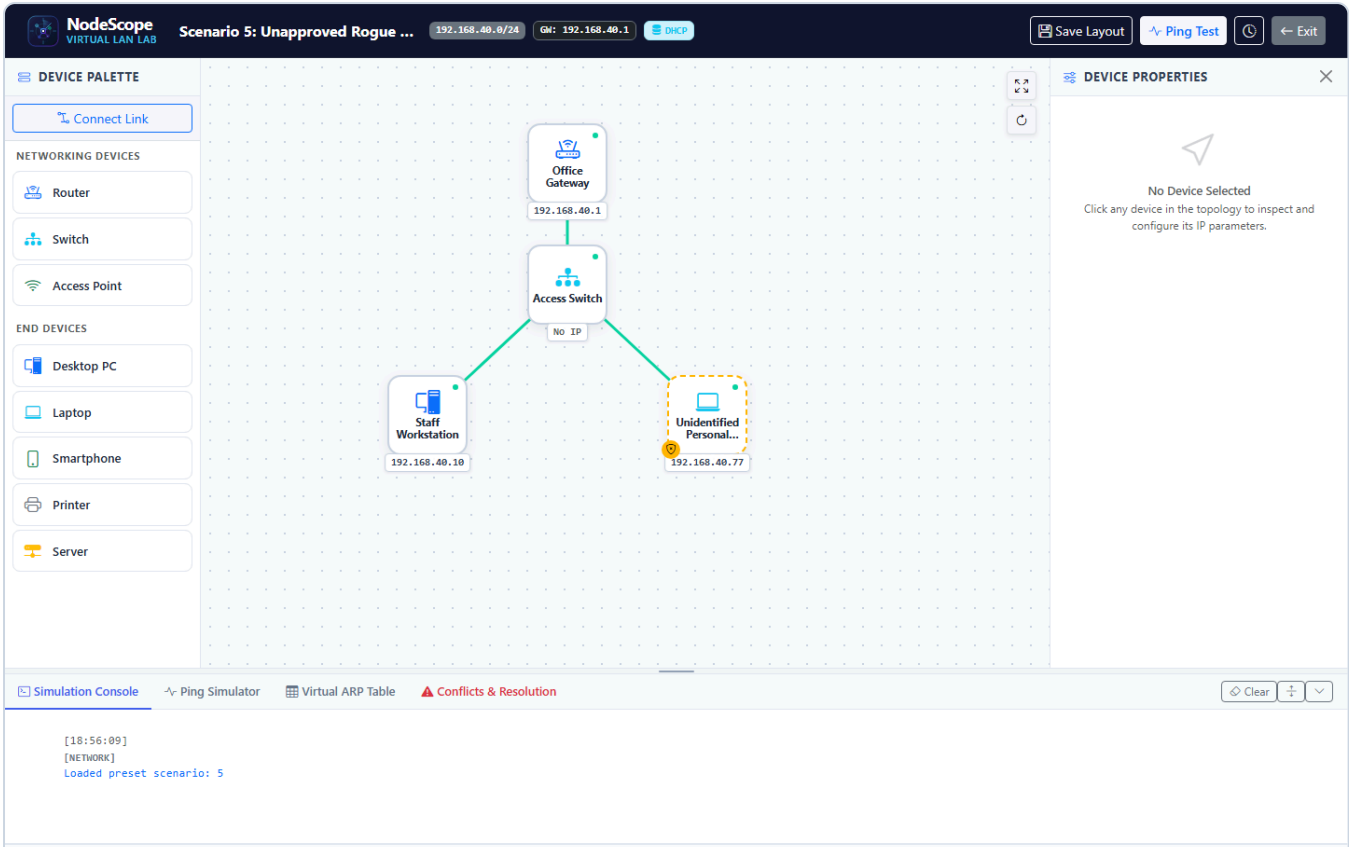


Figure 5.4: Unauthorized Rogue Device identified with dashed warning borders and Network Access Control inspection.

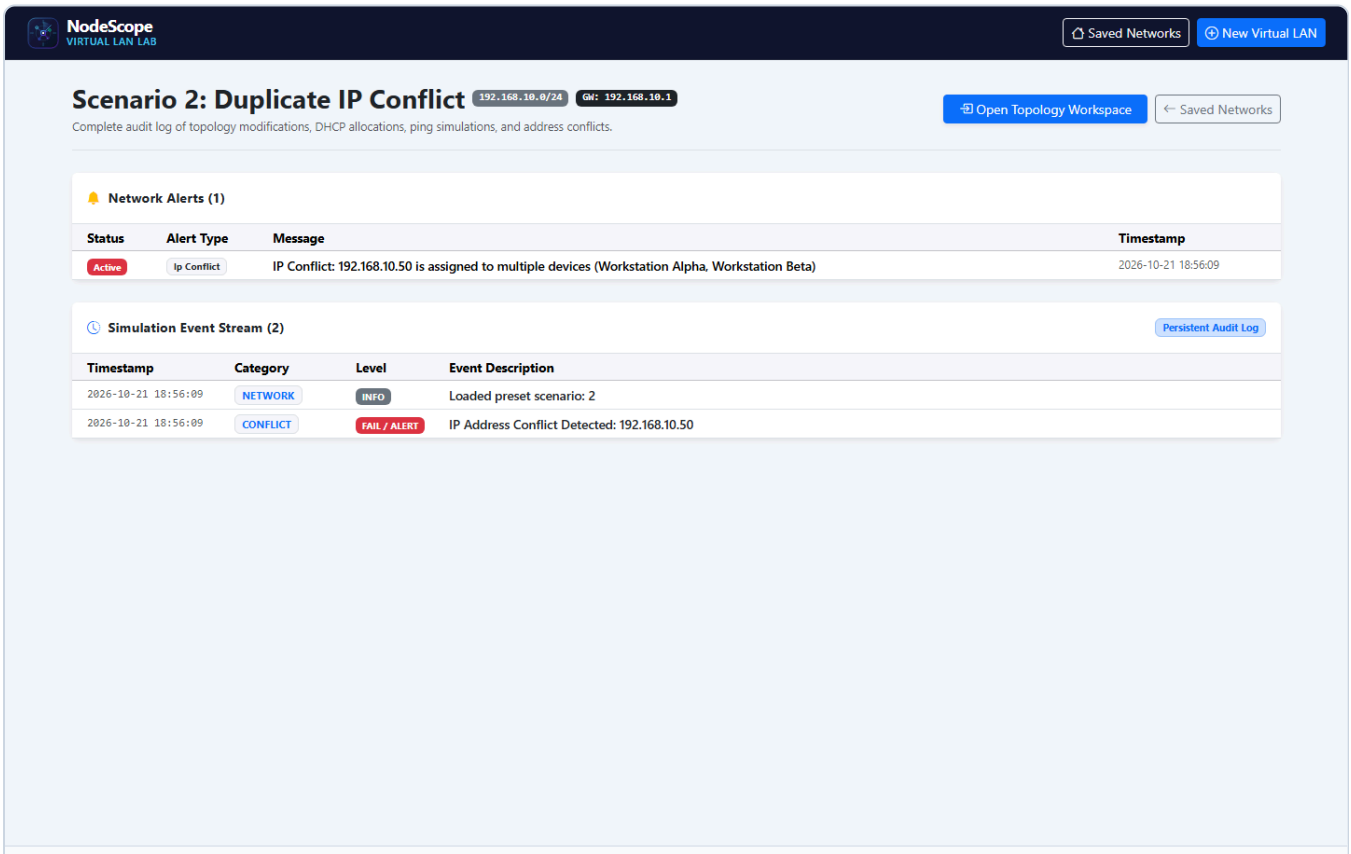


Figure 5.5: Persistent Activity Audit Stream tracking DHCP leases, pings, and security alerts.

6. Software Testing & Verification Suite

The simulator is rigorously verified through an automated test suite implemented in **Pytest**. The test suite executes in in-memory SQLite (`:memory:`), requiring zero network access:

Test Module	Tests	Target Validation Logic	Result
`test_sim_addressing.py`	9 Tests	CIDR parsing, netmask calculation, reserved IP detection, candidate search	PASS (100%)
`test_sim_devices.py`	1 Test	Device CRUD, MAC formatting, online toggling, and coordinate storage	PASS (100%)
`test_sim_dhcp.py`	1 Test	Sequential lease allocation, MAC binding, lease renewal, and pool exhaustion	PASS (100%)
`test_sim_connectivity.py`	1 Test	BFS graph traversal through multi-port switches and severed link detection	PASS (100%)
`test_sim_conflicts.py`	1 Test	Duplicate IP collision trigger, halo state, and candidate resolution	PASS (100%)
`test_sim_ping.py`	1 Test	5-step ICMP diagnostic breakdown, power check, and subnet matching	PASS (100%)
`test_sim_arp.py`	1 Test	Layer 2-to-3 resolution, active lease bindings, and ambiguous collision flags	PASS (100%)
`test_sim_scenarios.py`	1 Test	Pre-packaged verification of all 5 educational presentation scenarios	PASS (100%)
`test_sim_api.py`	1 Test	Flask JSON REST API integration, HTTP status codes, and JSON response models	PASS (100%)
`test_sim_events.py`	1 Test	Audit stream persistence and security triage alert logging	PASS (100%)

> **Test Execution Summary:** 18 passed in 0.57s on Python 3.14 / pytest-8.3.5. Zero warnings, zero regressions.

7. Viva Voce Examination Guide & Technical Defense

Top 10 High-Frequency Oral Exam Questions

Q1: What is the primary difference between a Hub and a Switch in your simulator?
Answer: In NodeScope, the Switch functions as an intelligent Layer 2 multi-port forwarding device that maintains topological path continuity for connected nodes without consuming a Layer 3 IP address. It forms the central vertex of our BFS star topology.

Q2: Why does an IP address collision break network communication?
Answer: When two hosts share the same IPv4 address, both reply to ARP Request broadcasts with different MAC addresses. This causes ARP cache ambiguity in neighboring hosts and causes switch MAC address tables to flap, leading to dropped frames.

Q3: How does your DHCP allocator ensure no overlapping IP addresses are assigned?
Answer: The DHCP engine queries both active static device allocations and existing unexpired leases. It iterates through the configured `[dhcp_start, dhcp_end]` pool and only grants an address if it is completely absent from the claimed set.

Q4: Explain the BFS algorithm used in the Ping Simulator.
Answer: We build an undirected adjacency list from the `connections` table where edges exist between connected devices. BFS traverses the graph from the source device queue. If an intermediate switch is offline (`is_online == 0`), its edges are pruned. Reachability succeeds if the destination device is dequeued.

Q5: Why did you choose Vanilla JavaScript and SVG over React or Canvas?
Answer: SVG represents connection links and devices as scalable DOM vector nodes with crisp rendering at any resolution. Vanilla JavaScript eliminates complex node_modules dependencies and npm build steps, keeping the complete project lightweight and startable with a single Python command.

8. Conclusion & Future Scope

NodeScope successfully achieves all stated engineering objectives. It provides a robust, educational, and visually engaging simulation platform for understanding computer networking fundamentals. By transforming abstract theoretical protocols into interactive visual workflows, NodeScope empowers students to experiment, make mistakes, observe failures, and resolve network problems safely on their own computers.

Future Enhancements

- **VLAN Tagging (802.1Q):** Simulating virtual LAN trunking and access port isolation on the core switch.
- **Static Inter-LAN Routing:** Introducing a dual-interface router allowing communication between separate subnets (e.g. `192.168.10.0/24` and `10.0.0.0/24`).
- **Network Performance Metrics:** Simulating link bandwidth, transmission queueing delay, and packet loss ratios.
